```
import './assets/main.css'
import './utils/publicAssetUrl.js'

import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
import { registerSpellCheckTaskBridge } from './utils/spellCheckTaskBridge.js'
import { installGlobalErrorLogger } from './utils/globalErrorLogger.js'
import { prepareDialogDisplayText } from './utils/dialogTextDisplay.js'

const app = createApp(App)

app.config.globalProperties.$cdt = (value) => prepareDialogDisplayText(value == null ? '' : String(value))

registerSpellCheckTaskBridge()
installGlobalErrorLogger(app)

app.use(router)

router.isReady().then(() => {
  app.mount('#app')
}).catch((e) => {
  console.error('Router ready failed:', e)
  app.mount('#app')
})


/**
 * WPS 加载项 public/ 静态资源 URL（与 dist 根目录下 index.html 同级的 images/ 等）。
 *
 * 策略（覆盖 wpsjs debug / 离线安装 / 在线子路径部署）：
 * 1. 优先用 PluginStorage「AddinBaseUrl」+ location 推导的插件根（Ribbon getImage 回调里 document.baseURI 往往不是加载项目录，不能放在最前）。
 * 2. 再用 document.baseURI + URL(relative, base)（任务窗格、对话框等正常 WebView）。
 * 3. 再否则当前页 location。
 * 4. 最后 import.meta.env.BASE_URL。
 */

function stripHash(href) {
  return String(href || '').split('#')[0]
}

function getStoredAddonBaseClean() {
  try {
    let s = window.Application?.PluginStorage?.getItem('AddinBaseUrl') || ''
    s = stripHash(s).replace(/\/?index\.html$/i, '').trim().replace(/\/+$/, '')
    return s
  } catch {
    return ''
  }
}
```

```
/**
 * 将当前页面对应的加载项目录写入 PluginStorage，供 Ribbon getImage 早于 Vue mount 使用。
 * 与 App.vue 中逻辑一致，集中在一处避免遗漏。
 */
export function syncAddonBaseUrlToPluginStorage() {
  try {
    if (typeof window === 'undefined' || !window.Application?.PluginStorage) return
    const href = (window.location.href || '').split('#')[0]
    let base = ''
    if (window.location.protocol === 'file:') {
      base = href.replace(/\/?index\.html$/i, '')
    } else {
      base =
        `${window.location.origin}${window.location.pathname || ''}`.replace(/\/?index\.html$/i, '') || ''
    }
    base = String(base || '').replace(/\/+$/, '')
    if (base) {
      base = `${base}/`
    }
    window.Application.PluginStorage.setItem('AddinBaseUrl', base || href)
  } catch {
    /* ignore */
  }
}

/**
 * 供 ribbon 等在无可靠 document 时使用：加载项目录，带尾部 /。
 */
export function getAddonRootHrefForRibbon() {
  if (typeof window === 'undefined') return ''
  const stored = getStoredAddonBaseClean()
  if (stored) {
    if (stored.startsWith('file:')) return stored.endsWith('/') ? stored : `${stored}/`
    if (/^https?:\/\/i.test(stored)) return stored.endsWith('/') ? stored : `${stored}/`
  }
  const href = stripHash(window.location?.href || '')
  const proto = window.location?.protocol || ''
  if (!href) return ''
  let base = ''
  if (proto === 'file:') {
    base = href.replace(/\/?index\.html$/i, '').replace(/\/+$/, '')
  } else {
    base = `${window.location.origin}${window.location.pathname || ''}`
      .replace(/\/?index\.html$/i, '')
      .replace(/\/+$/, '')
  }
  if (!base) return ''
  if (base.startsWith('file:')) return `${base}/`
  if (/^https?:\/\/i.test(base)) return `${base}/`
```

```
  return ''
}

/**
 * public 目录下资源的最终绝对或相对 URL。
 */
export function publicAssetUrl(relativePath) {
  const p = String(relativePath || '').replace(/^\/+/, '')
  if (!p) return ''

  if (/^(https?:\/\/|data:|file:)/i.test(p)) {
    return p
  }

  // 1) 插件根（PluginStorage + location）：Ribbon getImage 等场景 document.baseURI 不可靠，必须优先
  const rootFromAddon = getAddonRootHrefForRibbon()
  if (rootFromAddon) {
    try {
      return new URL(p, rootFromAddon).href
    } catch {
      return `${rootFromAddon.replace(/\/+$/, '/')}${p}`
    }
  }

  // 2) 与当前文档一致（对话框、任务窗格、ShowDialog 内 WebView）
  if (typeof document !== 'undefined' && document.baseURI) {
    try {
      return new URL(p, document.baseURI).href
    } catch {
      /* fall through */
    }
  }

  // 3) location
  if (typeof window !== 'undefined' && window.location?.href) {
    try {
      return new URL(p, stripHash(window.location.href)).href
    } catch {
      /* fall through */
    }
  }

  const fb = (import.meta.env.BASE_URL || './').replace(/\/?$/, '/')
  try {
    if (fb.startsWith('http://') || fb.startsWith('https://') || fb.startsWith('file:')) {
      return new URL(p, fb).href
    }
  } catch {
    /* ignore */
  }
```

```
  return `${fb}${p}`
}

if (typeof window !== 'undefined') {
  syncAddonBaseUrlToPluginStorage()
}

/** @deprecated 保留兼容；逻辑已并入 publicAssetUrl */
export function getAddonPublicDirectoryBase() {
  if (typeof document !== 'undefined' && document.baseURI) {
    try {
      const u = new URL('.', document.baseURI)
      return u.href
    } catch {
      /* ignore */
    }
  }
  const r = getAddonRootHrefForRibbon()
  return r || `${(import.meta.env.BASE_URL || './').replace(/\/?$/, '/')}`
}

import {
  startSpellCheckAllTask,
  startSpellCheckSelectionTask,
  stopSpellCheckTask
} from './spellCheckService.js'

const SPELL_CHECK_BRIDGE_KEY = '__ndSpellCheckTaskBridge'

function getCurrentWindow() {
  return typeof window !== 'undefined' ? window : null
}

export function registerSpellCheckTaskBridge() {
  const currentWindow = getCurrentWindow()
  if (!currentWindow) return
  currentWindow[SPELL_CHECK_BRIDGE_KEY] = {
    start(mode = 'all') {
      const runner = mode === 'selection' ? startSpellCheckSelectionTask : startSpellCheckAllTask
      const { taskId, promise } = runner()
      Promise.resolve(promise).catch((error) => {
        console.error('spellCheck bridge task failed:', error)
      })
      return { taskId }
    },
    stop(taskId) {
      return stopSpellCheckTask(taskId)
    }
  }
}
```

```
export function getSpellCheckTaskBridge() {
  const currentWindow = getCurrentWindow()
  if (!currentWindow) return null
  return currentWindow.opener?.[SPELL_CHECK_BRIDGE_KEY]
    || currentWindow.parent?.[SPELL_CHECK_BRIDGE_KEY]
    || currentWindow[SPELL_CHECK_BRIDGE_KEY]
    || null
}

import { ensureDir, getDefaultLogDir, getEffectiveDataDir, normalizeDataPath, pathJoin } from './dataPathSettings.js'

/** 已配置「数据路径」时，日志放在该目录下的子文件夹名 */
const LOG_SUBDIR = 'logs'

/** 单文件最大字节数（超出则同一天的下一个分片：YYYY-M-D.2.log …） */
const MAX_LOG_FILE_BYTES = 5 * 1024 * 1024

/** 保留最近 N 个自然日的日志文件，更早的删除 */
const RETENTION_DAYS = 30

/** 自动清理最小间隔，避免每次写日志都扫目录 */
const CLEANUP_INTERVAL_MS = 6 * 60 * 60 * 1000

/** 同一天最多分片数量（防止异常循环占满磁盘） */
const MAX_PARTS_PER_DAY = 500

/** 磁盘写入失败时写入 PluginStorage/localStorage，键名（JSON 字符串数组） */
const FALLBACK_STORAGE_KEY = 'NdErrorLogFallback'
const MAX_FALLBACK_LINES = 100

/**
 * 尽量解析 WPS Application（iframe / 多窗时可能在 parent、top 上）
 */
function getApplication() {
  if (typeof window === 'undefined') return null
  if (window.Application) return window.Application
  try {
    if (window.top && window.top !== window && window.top.Application) {
      return window.top.Application
    }
  } catch (e) {
    void e
  }
  if (window.opener?.Application) return window.opener.Application
  if (window.parent?.Application) return window.parent.Application
  try {
    let w = window.parent
    for (let i = 0; i < 6 && w; i++) {
      if (w.Application) return w.Application
```

```
      if (w === w.parent) break
      w = w.parent
    }
  } catch (e) {
    void e
  }
  return null
}

let lastCleanupAt = 0

/** 本次 writeLogLine 未写盘原因：'no_fs' | '' */
let lastDiskSkipReason = ''

let warnedFileSystemUnavailable = false

function warnFileSystemUnavailableOnce() {
  if (warnedFileSystemUnavailable) return
  warnedFileSystemUnavailable = true
  const app = getApplication()
  if (!app) {
    console.warn(
      '[GlobalErrorLogger] 无 WPS Application.FileSystem（常见于用浏览器直接打开本地调试页）。异常仍会写入 NdErrorLogFallback；在 WPS 内打开加载项后可写磁盘日志。'
    )
  } else {
    console.warn('[GlobalErrorLogger] Application 已找到但 FileSystem 不可用，异常已写入 NdErrorLogFallback。')
  }
}

function isAbsoluteFsPath(p) {
  if (!p || typeof p !== 'string') return false
  const s = p.trim()
  if (s.startsWith('/')) return true
  return /^[A-Za-z]:[\\/]/.test(s)
}

/** 将相对路径转为绝对路径（WPS FileSystem 对相对路径常写入失败） */
function resolveAbsoluteFsPath(fs, p) {
  if (!p) return p
  const s = String(p).trim()
  if (isAbsoluteFsPath(s)) return s.replace(/[/\\]+$/, '')
  if (fs && typeof fs.absoluteFilePath === 'function') {
    try {
      const abs = fs.absoluteFilePath(s)
      if (abs && isAbsoluteFsPath(String(abs))) {
        return String(abs).replace(/[/\\]+$/, '')
      }
    } catch (e) {
      void e
    }
```

```
  }
  return s
}

/** WPS FileSystem 在 macOS 上常要求原生分隔符 */
function normalizeFsPathForWps(fs, p) {
  if (!p || !fs) return p
  if (typeof fs.toNativeSeparators === 'function') {
    try {
      return String(fs.toNativeSeparators(p))
    } catch (e) {
      void e
    }
  }
  return p
}

/** 启动探测写入后删除，用于确认目录可写 */
const LOG_DIR_PROBE_FILE = '.chayuan_log_dir_probe'

function tryEnsureLogDir(fs, dirPath) {
  let p = normalizeFsPathForWps(fs, resolveAbsoluteFsPath(fs, dirPath))
  ensureDir(fs, p)
  if (typeof fs.mkdirSync === 'function') {
    try {
      fs.mkdirSync(p)
    } catch (e) {
      void e
    }
  }
  if (typeof fs.existsSync === 'function' && !fs.existsSync(p)) {
    ensureDir(fs, p)
  }
  return p
}

/**
 * 探测目录是否可写：写入短文件再删除（与真实日志同一 API）
 */
function probeLogDirWritable(fs, logsDir) {
  if (!logsDir || !fs) return false
  const probePath = normalizeFsPathForWps(fs, pathJoin(logsDir, LOG_DIR_PROBE_FILE))
  const write = fs.writeFileString || fs.WriteFile
  if (typeof write !== 'function') return false
  try {
    if (!write.call(fs, probePath, 'ok')) return false
    try {
      if (typeof fs.unlinkSync === 'function') fs.unlinkSync(probePath)
      else if (typeof fs.Remove === 'function') fs.Remove(probePath)
    } catch (e) {
```

```
      void e
    }
    return true
  } catch (e) {
    void e
    return false
  }
}

/** 启动时：对每个候选路径解析为绝对路径、递归创建目录、探测可写 */
function prepareLogDirectoriesOnInstall() {
  const fs = getApplication()?.FileSystem
  if (!fs) return
  const candidates = collectLogDirCandidates()
  const tried = []
  for (const raw of candidates) {
    if (!raw) continue
    let dir = tryEnsureLogDir(fs, raw)
    dir = resolveAbsoluteFsPath(fs, dir)
    dir = normalizeFsPathForWps(fs, dir)
    tryEnsureLogDir(fs, dir)
    const ok = probeLogDirWritable(fs, dir)
    tried.push({ dir, ok })
    if (ok) {
      return
    }
  }
  if (tried.length > 0) {
    console.warn('[GlobalErrorLogger] 启动时未能创建或可写任一日志目录（已尝试创建路径）', {
      tried,
      hint: '请确认「数据路径」所在磁盘可写，或检查 WPS 对路径的访问权限'
    })
  }
}

/**
 * 主目录失败时依次尝试：用户配置 logs → 系统默认日志目录 → %TEMP%/chayuan/logs
 */
function collectLogDirCandidates() {
  const seen = new Set()
  const out = []
  const push = (p) => {
    if (!p || typeof p !== 'string') return
    const t = p.trim()
    if (!t || seen.has(t)) return
    seen.add(t)
    out.push(t)
  }
  try {
    push(getLogsDir())
```

```
  } catch (e) {
    void e
  }
  try {
    push(getDefaultLogDir())
  } catch (e) {
    void e
  }
  try {
    const app = getApplication()
    if (app?.Env && typeof app.Env.GetTempPath === 'function') {
      const raw = String(app.Env.GetTempPath() || '')
        .replace(/^file:\/\/i, '')
        .replace(/[/\\]+$/g, '')
      if (raw) push(pathJoin(raw, 'chayuan', 'logs'))
    }
  } catch (e) {
    void e
  }
  return out
}

/**
 * 磁盘不可用时保留最近若干条 JSON 行，便于排查（闪退时仍可能丢失）
 */
function appendFallbackStorage(line) {
  try {
    const app = getApplication()
    let rows = []
    try {
      const raw = app?.PluginStorage?.getItem(FALLBACK_STORAGE_KEY)
      if (raw) {
        const parsed = JSON.parse(raw)
        if (Array.isArray(parsed)) rows = parsed
      }
    } catch (e) {
      void e
    }
    if (rows.length === 0 && typeof localStorage !== 'undefined') {
      try {
        const raw = localStorage.getItem(FALLBACK_STORAGE_KEY)
        if (raw) {
          const parsed = JSON.parse(raw)
          if (Array.isArray(parsed)) rows = parsed
        }
      } catch (e) {
        void e
      }
    }
    rows.push(line)
```

```
    while (rows.length > MAX_FALLBACK_LINES) rows.shift()
    const out = JSON.stringify(rows)
    try {
      app?.PluginStorage?.setItem(FALLBACK_STORAGE_KEY, out)
    } catch (e) {
      void e
    }
    try {
      if (typeof localStorage !== 'undefined') localStorage.setItem(FALLBACK_STORAGE_KEY, out)
    } catch (e) {
      void e
    }
  } catch (e) {
    void e
  }
}

/** 供调试：读取备用缓冲中的原始 JSON 行 */
export function getErrorLogFallbackLines() {
  try {
    const raw =
      getApplication()?.PluginStorage?.getItem(FALLBACK_STORAGE_KEY) ||
      (typeof localStorage !== 'undefined' ? localStorage.getItem(FALLBACK_STORAGE_KEY) : null)
    if (!raw) return []
    const parsed = JSON.parse(raw)
    return Array.isArray(parsed) ? parsed : []
  } catch (e) {
    return []
  }
}

function utf8ByteLength(s) {
  if (typeof TextEncoder !== 'undefined') {
    return new TextEncoder().encode(s).length
  }
  try {
    return unescape(encodeURIComponent(s)).length
  } catch (e) {
    return String(s).length
  }
}

function normalizeValue(value) {
  if (value instanceof Error) {
    return {
      name: value.name,
      message: value.message,
      stack: value.stack
    }
  }
```

```
  if (value && typeof value === 'object') {
    try {
      return JSON.parse(JSON.stringify(value))
    } catch (e) {
      return String(value)
    }
  }
  return value
}

function formatDateBaseName(date = new Date()) {
  const y = date.getFullYear()
  const m = date.getMonth() + 1
  const d = date.getDate()
  return { y, m, d, base: `${y}-${m}-${d}` }
}

function partFileName(base, part) {
  if (part <= 1) return `${base}.log`
  return `${base}.${part}.log`
}

/** 当前实际写入错误日志的目录（与运行时一致） */
export function getEffectiveLogDir() {
  const configured = getEffectiveDataDir()
  if (configured) {
    return pathJoin(configured, LOG_SUBDIR)
  }
  return getDefaultLogDir()
}

/**
 * 根据设置页「数据路径」输入框内容，预览错误日志所在目录（留空则与未配置数据路径时的系统默认日志目录一致）
 */
export function getErrorLogDirectoryForDataPath(dataPathField) {
  const t = (dataPathField || '').trim()
  if (t) {
    return pathJoin(normalizeDataPath(t), LOG_SUBDIR)
  }
  return getDefaultLogDir()
}

function getLogsDir() {
  return getEffectiveLogDir()
}

function safeStringify(payload) {
  try {
    return JSON.stringify(payload)
  } catch (e) {
```

```
    return JSON.stringify({
      type: 'logger_serialize_failed',
      reason: String(e?.message || e)
    })
  }
}

function getFileSize(fs, path) {
  try {
    if (typeof fs.stat === 'function') {
      const st = fs.stat(path)
      if (st && typeof st.size === 'number') return st.size
    }
  } catch (e) {
    void e
  }
  return 0
}

function fileExists(fs, path) {
  try {
    if (typeof fs.existsSync === 'function') return fs.existsSync(path)
    if (typeof fs.Exists === 'function') return fs.Exists(path)
  } catch (e) {
    void e
  }
  return false
}

function appendToFile(fs, path, line) {
  const p = normalizeFsPathForWps(fs, path)
  try {
    const exists = fileExists(fs, p)
    const payload = exists ? `\n${line}` : line
    // 部分 WPS 版本对「不存在文件」调用 AppendFile 会失败，仅对已存在文件追加
    if (exists && typeof fs.AppendFile === 'function') {
      if (fs.AppendFile(p, payload)) return true
    }
    const readFile = fs.readFileString || fs.ReadFile
    let old = ''
    if (exists && typeof readFile === 'function') {
      try {
        old = String(readFile.call(fs, p) || '')
      } catch (e) {
        old = ''
      }
    }
    const content = old ? `${old}${payload}` : line
    if (typeof fs.writeFileString === 'function' && fs.writeFileString(p, content)) return true
    if (typeof fs.WriteFile === 'function' && fs.WriteFile(p, content)) return true
```

```
    const writeAny = fs.writeFileString || fs.WriteFile
    if (typeof writeAny === 'function') {
      return !!writeAny.call(fs, p, content)
    }
    return false
  } catch (e) {
    return false
  }
}

function removeFile(fs, path) {
  try {
    if (typeof fs.unlinkSync === 'function') return fs.unlinkSync(path)
    if (typeof fs.Remove === 'function') {
      fs.Remove(path)
      return true
    }
  } catch (e) {
    void e
  }
  return false
}

/**
 * 选择当日可写入的分片路径：优先已有未满文件，否则新建下一分片
 */
function resolveLogPathForLine(fs, logsDir, line) {
  const { base } = formatDateBaseName()

  for (let part = 1; part <= MAX_PARTS_PER_DAY; part++) {
    const name = partFileName(base, part)
    const full = normalizeFsPathForWps(fs, pathJoin(logsDir, name))
    const exists = fileExists(fs, full)
    if (!exists) {
      return full
    }
    const size = getFileSize(fs, full)
    const appendPayload = size > 0 ? `\n${line}` : line
    const addBytes = utf8ByteLength(appendPayload)
    if (size + addBytes <= MAX_LOG_FILE_BYTES) {
      return full
    }
  }
  return null
}

const LOG_NAME_RE = /^(\d{4})-(\d{1,2})-(\d{1,2})(?:\.(\d+))?\.log$/

function parseLogFileDate(name) {
  const m = String(name).match(LOG_NAME_RE)
```

```
  if (!m) return null
  const y = Number(m[1])
  const mo = Number(m[2])
  const d = Number(m[3])
  if (!y || !mo || !d) return null
  return new Date(y, mo - 1, d)
}

function maybeCleanupOldLogs(fs, logsDir) {
  const now = Date.now()
  if (now - lastCleanupAt < CLEANUP_INTERVAL_MS) return
  lastCleanupAt = now

  if (!logsDir || typeof fs.readdirSync !== 'function') return

  let names = []
  try {
    names = fs.readdirSync(logsDir) || []
  } catch (e) {
    return
  }
  if (!Array.isArray(names) || names.length === 0) return

  const today = new Date()
  today.setHours(0, 0, 0, 0)

  for (const name of names) {
    if (!name || typeof name !== 'string' || !name.endsWith('.log')) continue
    const fileDate = parseLogFileDate(name)
    if (!fileDate) continue
    fileDate.setHours(0, 0, 0, 0)
    const diffDays = Math.floor((today - fileDate) / 86400000)
    if (diffDays > RETENTION_DAYS) {
      removeFile(fs, pathJoin(logsDir, name))
    }
  }
}

function writeLogLine(line) {
  lastDiskSkipReason = ''
  try {
    const app = getApplication()
    const fs = app?.FileSystem
    if (!fs) {
      appendFallbackStorage(line)
      lastDiskSkipReason = 'no_fs'
      warnFileSystemUnavailableOnce()
      return false
    }
```

```
    const candidates = collectLogDirCandidates()
    if (candidates.length === 0) {
      appendFallbackStorage(line)
      console.error('[GlobalErrorLogger] 日志未写入磁盘：无法解析日志目录')
      return false
    }

    const tried = []
    for (const rawDir of candidates) {
      let logsDir = tryEnsureLogDir(fs, rawDir)
      logsDir = resolveAbsoluteFsPath(fs, logsDir)

      const path = resolveLogPathForLine(fs, logsDir, line)
      if (!path) {
        tried.push({ dir: logsDir, error: 'no_shard' })
        continue
      }

      const targetPath = resolveAbsoluteFsPath(fs, path)
      const ok = appendToFile(fs, targetPath, line)
      if (ok) {
        maybeCleanupOldLogs(fs, logsDir)
        return true
      }
      tried.push({ dir: logsDir, file: targetPath, error: 'write_failed' })
    }

    appendFallbackStorage(line)
    console.error('[GlobalErrorLogger] 日志未写入磁盘：已尝试所有候选目录仍失败', {
      tried,
      hint: '若主目录不可写，请检查系统默认日志目录或 %TEMP%/chayuan/logs'
    })
    return false
  } catch (e) {
    appendFallbackStorage(line)
    console.error('[GlobalErrorLogger] 日志写入异常', e)
    return false
  }
}

function logError(type, payload) {
  const line = safeStringify({
    ts: new Date().toISOString(),
    type,
    ...payload
  })

  const ok = writeLogLine(line)
  if (!ok && lastDiskSkipReason !== 'no_fs') {
    console.error('[GlobalErrorLogger] 日志写入失败', line)
```

```
  }
}

export function installGlobalErrorLogger(app) {
  if (!app) return

  try {
    prepareLogDirectoriesOnInstall()
    const fs = getApplication()?.FileSystem
    const dir = getLogsDir()
    if (fs && dir) {
      const abs = tryEnsureLogDir(fs, dir)
      maybeCleanupOldLogs(fs, abs)
    }
  } catch (e) {
    void e
  }

  try {
    writeLogLine(
      safeStringify({
        ts: new Date().toISOString(),
        type: 'logger_boot',
        logsDir: getLogsDir(),
        ua: typeof navigator !== 'undefined' ? navigator.userAgent : ''
      })
    )
  } catch (e) {
    void e
  }

  app.config.errorHandler = (err, instance, info) => {
    logError('vue_error', {
      info: String(info || ''),
      error: normalizeValue(err),
      component: instance?.$options?.name || instance?.type?.name || ''
    })
    console.error('[Vue Error]', info, err)
    // 不在此处弹层：避免渲染期错误 → 弹窗 → 再次触发 Vue 更新形成递归。
  }

  // 仅记录日志，不在此处弹 DOM/剪贴板：在 CrBrowserMain 上与 WPS 桥并发时曾观察到宿主 abort（与 ksolite 崩溃报告一致）。
  window.addEventListener(
    'error',
    (event) => {
      logError('window_error', {
        message: String(event?.message || ''),
        filename: String(event?.filename || ''),
        lineno: Number(event?.lineno || 0),
        colno: Number(event?.colno || 0),
```

```
        error: normalizeValue(event?.error)
      })
      try {
        console.error('[window.error]', event?.message, event?.error)
      } catch (e) {
        void e
      }
    },
    true
  )

  window.addEventListener('unhandledrejection', (event) => {
    logError('unhandled_rejection', {
      reason: normalizeValue(event?.reason)
    })
    try {
      console.error('[unhandledrejection]', event?.reason)
    } catch (e) {
      void e
    }
  })
}

/**
 * Strip markdown-style punctuation from text shown in assistant dialog UI
 * so users do not see raw * - # etc. from model output.
 */
export function sanitizeDialogDisplaySymbols(text) {
  if (text == null) return ''
  let s = String(text)
  if (!s) return ''
  s = s.replace(/\[([^\]]*)\]\([^)]*\)/g, '$1')
  s = s.replace(/\*/g, '')
  s = s.replace(/`/g, '')
  s = s.replace(/^#{1,6}\s*/gm, '')
  s = s.replace(/^\s*([-*_])(?:\s*\1){2,}\s*$/gm, '')
  s = s.replace(/^(\s*)[-*+]\s+/gm, '$1')
  s = s.replace(/^(\s*)>\s?/gm, '$1')
  s = s.replace(/~{2}/g, '')
  s = s.replace(/_{2,}/g, '')
  return s
}

function toHalfWidthDigits(s) {
  return s.replace(/[ 0 - 9 ]/g, (c) => String.fromCharCode(c.charCodeAt(0) - 0xff10 + 48))
}

/**
 * Normalize LLM-style numbered lists (e.g. "1、xx 2、xx" on one line) into line breaks
 * so paste into Word / insert document gets one item per paragraph. Idempotent enough for typical output.
```

```
 */
export function normalizeLlmEnumerationLayout(text) {
  if (text == null) return ''
  let s = toHalfWidthDigits(String(text))
  if (!s.trim()) return s

  // After sentence punctuation, start numbered item on a new line: "如下：1、"
  s = s.replace(/([。 ； ： ： ！ ？])(\s*)(\d{1,2})([、 ])\s*/g, '$1\n$3$4 ')

  // (1) (2) style with following text
  s = s.replace(/(?<=[^\n\r])(\s*)(\([1-9]\d?\))\s+(?=[\u4e00-\u9fffA-Za-z「【（])/g, '\n$2 ')

  // Mid-line " 2、" " 3、" (space + number +顿号)
  s = s.replace(/(?<=[^\n\r])(\s+)(\d{1,2})([、])\s*/g, '\n$2$3 ')

  // Mid-line " 2. " western numbering before Chinese/Latin (avoid 3.14)
  s = s.replace(/(?<=[^\n\r])(\s+)(\d{1,2})[. . ]\s+(?=[\u4e00-\u9fffA-Za-z「【（])/g, '\n$2、 ')

  // Line-start "1. " -> "1、"
  s = s.replace(/(^|\n)(\s*)(\d{1,2})[. . ]\s+/g, '$1$2$3、 ')

  // Trim excessive blank lines (keep at most one blank line)
  s = s.replace(/\n{3,}/g, '\n\n')

  s = s
    .split('\n')
    .map((line) => line.trimEnd())
    .join('\n')

  return s.trim()
}

/** Display pipeline: list layout then symbol strip (for chat bubbles & assistant summaries). */
export function prepareDialogDisplayText(text) {
  let s = String(text ?? '')
  if (!s) return ''
  s = normalizeLlmEnumerationLayout(s)
  s = sanitizeDialogDisplaySymbols(s)
  return s
}

<template>
  <RouterView :key="$route.fullPath" />
</template>



<script>
import { ref, onMounted, watch } from 'vue'
import { useRoute } from 'vue-router'
```

```
import ribbon from './components/ribbon.js'
import { syncAddonBaseUrlToPluginStorage } from './utils/publicAssetUrl.js'
import { schedulePreloadAiAssistantRouteChunk } from './utils/preloadAiAssistantChunk.js'

const DIALOG_ROUTES = ['/settings', '/dialog', '/ad-popup', '/manual-col-width', '/dialog-delete-text',
  '/append-replace-text', '/dialog-first-col-style', '/dialog-uniform-image-format', '/table-caption',
  '/document-declassify-dialog', '/document-declassify-restore-dialog', '/template-import-dialog',
  '/template-export-dialog', '/template-download-dialog', '/document-template-import',
  '/template-field-extract-dialog', '/form-content-preview', '/form-audit-dialog',
  '/unused-styles-cleaner-dialog', '/style-statistics-dialog', '/ai-assistant', '/task-orchestration',
  '/about-chayuan']

export default {
  setup() {
    const message = ref('你好，wps加载项')
    const route = useRoute()

    function updateDialogPageClass() {
      const path = (route.path || '').replace(/\/$/, '') || '/'
      const isDialog = DIALOG_ROUTES.some(r => path === r || path.startsWith(r + '/'))
      document.body.classList.toggle('dialog-page', isDialog)
      const appEl = document.getElementById('app')
      if (appEl) appEl.classList.toggle('dialog-page', isDialog)
    }

    onMounted(() => {
      window.ribbon = ribbon
      updateDialogPageClass()
      syncAddonBaseUrlToPluginStorage()
      schedulePreloadAiAssistantRouteChunk()
    })

    watch(() => route.path, updateDialogPageClass, { immediate: true })

    return {
      message
    }
  }
}
</script>


import { createRouter, createWebHashHistory } from 'vue-router'
//import HomeView from '../views/HomeView.vue'

const router = createRouter({
  history:  createWebHashHistory(''),
  routes: [
    {
      path: '/',
```

```
  name: '默认页',
  component: () => import('../components/Root.vue')
},
{
  path: '/dialog',
  name: '对话框',
  component: () => import('../components/Dialog.vue')
},
{
  path: '/taskpane',
  name: '任务窗格',
  component: () => import('../components/TaskPane.vue')
},
{
  path: '/taskpane-top',
  name: '顶部任务窗口',
  component: () => import('../components/TaskPaneTop.vue')
},
{
  path: '/taskpane-bottom',
  name: '底部任务窗口',
  component: () => import('../components/TaskPaneBottom.vue')
},
{
  path: '/taskpane-left',
  name: '左侧任务窗口',
  component: () => import('../components/TaskPaneLeft.vue')
},
{
  path: '/taskpane-right',
  name: '规则库',
  component: () => import('../components/TaskPaneRight.vue')
},
{
  path: '/popup',
  name: '任务清单',
  component: () => import('../components/Popup.vue')
},
{
  path: '/task-orchestration',
  name: '任务编排',
  component: () => import('../components/TaskOrchestrationDialog.vue')
},
{
  path: '/spell-check-dialog',
  name: '拼写与语法检查',
  component: () => import('../components/TaskProgressDialog.vue')
},
{
  path: '/task-progress-dialog',
```

```
  name: '任务进度',
  component: () => import('../components/TaskProgressDialog.vue')
},
{
  path: '/manual-col-width',
  name: '手动列宽',
  component: () => import('../components/ManualColWidth.vue')
},
{
  path: '/dialog-delete-text',
  name: '删除文字所在行/列',
  component: () => import('../components/DeleteTextDialog.vue')
},
{
  path: '/append-replace-text',
  name: '追加或替换文字',
  component: () => import('../components/AppendReplaceText.vue')
},
{
  path: '/dialog-first-col-style',
  name: '第一列指定样式',
  component: () => import('../components/FirstColStyleDialog.vue')
},
{
  path: '/table-caption',
  name: '添加或修改题注',
  component: () => import('../components/TableCaptionDialog.vue')
},
{
  path: '/dialog-uniform-image-format',
  name: '统一图像格式',
  component: () => import('../components/UniformImageFormatDialog.vue')
},
{
  path: '/document-declassify-dialog',
  name: '文档脱密',
  component: () => import('../components/DocumentDeclassifyDialog.vue')
},
{
  path: '/document-declassify-restore-dialog',
  name: '密码复原',
  component: () => import('../components/DocumentDeclassifyRestoreDialog.vue')
},
{
  path: '/template-create',
  name: '规则制作',
  component: () => import('../components/TemplateCreate.vue')
},
{
  path: '/template-form-dialog',
```

```
  name: '添加/修改表单项',
  component: () => import('../components/TemplateFormDialog.vue')
},
{
  path: '/template-field-extract-dialog',
  name: '智能提取',
  component: () => import('../components/TemplateFieldExtractDialog.vue')
},
{
  path: '/form-content-preview',
  name: '表单内容预览',
  component: () => import('../components/FormContentPreview.vue')
},
{
  path: '/form-audit-dialog',
  name: '文档审计',
  component: () => import('../components/FormAuditDialog.vue')
},
{
  path: '/form-edit-dialog',
  name: '表单编辑',
  component: () => import('../components/FormEditDialog.vue')
},
{
  path: '/template-export-dialog',
  name: '导出规则',
  component: () => import('../components/TemplateExportDialog.vue')
},
{
  path: '/template-import-dialog',
  name: '规则导入',
  component: () => import('../components/TemplateImportDialog.vue')
},
{
  path: '/document-template-import',
  name: '导入模板',
  component: () => import('../components/DocumentTemplateImport.vue')
},
{
  path: '/template-download-dialog',
  name: '下载模板',
  component: () => import('../components/TemplateDownloadDialog.vue')
},
{
  path: '/settings',
  name: '设置',
  component: () => import('../components/SettingsDialog.vue')
},
{
  path: '/style-statistics-dialog',
```

```
      name: '样式使用统计',
      component: () => import('../components/StyleStatisticsDialog.vue')
    },
    {
      path: '/unused-styles-cleaner-dialog',
      name: '未使用样式清理',
      component: () => import('../components/UnusedStylesCleanerDialog.vue')
    },
    {
      path: '/ai-assistant',
      name: 'AI 助手',
      component: () => import('../components/AIAssistantDialog.vue')
    },
    {
      path: '/about-chayuan',
      name: '关于察元',
      component: () => import('../components/AboutChayuanPage.vue')
    }
  ]
})

export default router

<template>
  <div>
    <h1>{{ msg }}</h1>
  </div>
</template>

<script>
import ribbon from './ribbon.js'

export default {
  name: 'HelloWps',
  data() {
    return {
      msg: '欢迎来到wps加载项的世界!'
    }
  },
  mounted() {
    if (!window.Application?.ShowDialog || typeof ribbon.showAIAssistantDialog !== 'function') {
      return
    }
    // 等首页完成挂载后再自动拉起欢迎窗口，避免启动瞬间竞争导致弹窗丢失。
    window.setTimeout(() => {
      try {
        ribbon.showAIAssistantDialog()
      } catch (error) {
        console.warn('自动打开察元 AI 助手失败:', error)
      }
```

```
    }, 180)
  }
}
</script>

<!-- Add "scoped" attribute to limit CSS to this component only -->



//在后续的wps版本中，wps的所有枚举值都会通过wps.Enum对象来自动支持，现阶段先人工定义
var WPS_Enum = {
  msoCTPDockPositionLeft: 0,
  msoCTPDockPositionRight: 2,
  // 文档保护类型
  wdNoProtection: -1,
  wdAllowOnlyFormFields: 2,
  // 内容控件类型与外观
  wdContentControlText: 1,
  wdContentControlHidden: 2
}

function GetUrlPath() {
  // 在本地网页的情况下获取路径（与 index.html 同目录）
  if (window.location.protocol === 'file:') {
    const path = String(window.location.href || '').split('#')[0]
    return path.substring(0, path.lastIndexOf('/'))
  }

  // http(s)：包含 pathname，子路径部署时 ShowDialog 与资源根才正确
  const { protocol, hostname, port, pathname } = window.location
  const portPart = port ? `:${port}` : ''
  let p = pathname || '/'
  p = p.replace(/\/?index\.html$/i, '') || '/'
  if (p.length > 1 && p.endsWith('/')) p = p.slice(0, -1)
  const pathPart = p === '/' ? '' : p
  return `${protocol}//${hostname}${portPart}${pathPart}`
}

/**
 * Vue 使用 hash 路由。离线 file:// 下必须带入口文件名，否则会变成「目录 + 路由名」的假路径，
 * ShowDialog / CreateTaskPane 无法加载 index.html，页面与图标资源全部失效。
 * 例：file:///.../chayuan_1.0.1/index.html#/ai-assistant
 */
function GetRouterHash() {
  if (window.location.protocol === 'file:') {
    return '/index.html#'
  }

  return '/#'
}
```

```
/**
 * 从 PluginStorage 的 AddinBaseUrl（目录，可含尾部 /）拼出与 hash 路由一致的 SPA URL。
 * @param {string} base - AddinBaseUrl
 * @param {string} hashRouteAndQuery - 如 dialog-delete-text?mode=row（无开头的 /）
 */
function addonSpaUrlFromStorageBase(base, hashRouteAndQuery) {
  const clean = String(base || '')
    .replace(/#.*$/, '')
    .replace(/\/index\.html$/i, '')
    .replace(/\/+$/, '')
  const path = String(hashRouteAndQuery || '').replace(/^\//, '')
  if (!path || !clean) return ''
  return clean.startsWith('file:')
    ? `${clean}/index.html#/${path}`
    : `${clean}/#/${path}`
}

export default {
  WPS_Enum,
  GetUrlPath,
  GetRouterHash,
  addonSpaUrlFromStorageBase
}

function openOfficeFileFromSystemDemo(param) {
  let jsonObj = typeof param == 'string' ? JSON.parse(param) : param
  alert('从业务系统传过来的参数为：' + JSON.stringify(jsonObj))
  return { wps加载项项返回: jsonObj.filepath + ', 这个地址给的不正确' }
}

function InvokeFromSystemDemo(param) {
  let jsonObj = typeof param == 'string' ? JSON.parse(param) : param
  let handleInfo = jsonObj.Index
  switch (handleInfo) {
    case 'getDocumentName': {
      let docName = ''
      if (window.Application.ActiveDocument) {
        docName = window.Application.ActiveDocument.Name
      }

      return { 当前打开的文件名为: docName }
    }

    case 'newDocument': {
      let newDocName = ''
      let doc = window.Application.Documents.Add()
      newDocName = doc.Name

      return { 操作结果: '新建文档成功，文档名为：' + newDocName }
    }
```

```
    case 'OpenFile': {
      let filePath = jsonObj.filepath
      window.Application.Documents.OpenFromUrl(filePath)
      return { 操作结果: '打开文件成功' }
    }
  }

  return { 其它xxx: '' }
}

export default {
  openOfficeFileFromSystemDemo,
  InvokeFromSystemDemo
}

import Util from './js/util.js'
import SystemDemo from './js/systemdemo.js'
import * as XLSX from 'xlsx'
import { loadRulesFromDoc, saveRulesToDoc } from '../utils/templateRules.js'
import { setNewFileMarker, isNewFile } from '../utils/documentTemplates.js'
import { getModelLogoPath } from '../utils/modelLogos.js'
import { isRibbonModelEnabled } from '../utils/modelSettings.js'
import {
  getAssistantsForDisplayLocation,
  getAssistantDisplayEntry,
  isAssistantDisplayedInLocation
} from '../utils/assistantSettings.js'
import { getRibbonCompatibleAssistantIcon } from '../utils/assistantIcons.js'
import {
  CONTEXT_MENU_DYNAMIC_SLOT_COUNT,
  FIXED_MAIN_ASSISTANT_IDS,
  GROUP_CONTROL_ASSISTANT_MAP,
  getAssistantResolvedIcon,
  MAIN_CONTROL_ASSISTANT_MAP,
  RIBBON_DYNAMIC_SLOT_COUNT
} from '../utils/assistantRegistry.js'
import {
  startAssistantTask,
  getAssistantLaunchInfo,
  getTranslateLanguageByControlId
} from '../utils/assistantTaskRunner.js'
import { getDocumentText, getSelectedText } from '../utils/documentActions.js'
import { getSelectionContextSnapshot } from '../utils/documentContext.js'
import {
  invalidateDeclassifyRibbonControls,
  isDocumentDeclassified
} from '../utils/documentDeclassifyStore.js'
import { reopenExistingAIAssistantWindow } from '../utils/aiAssistantWindowManager.js'
import { MODEL_GROUPS, getDefaultModelsFlat } from '../utils/defaultModelGroups.js'
```

```
import { focusExistingSettingsWindow, openSettingsWindow } from '../utils/settingsWindowManager.js'
import { focusExistingTaskListWindow } from '../utils/taskListWindowManager.js'
import { focusExistingTaskOrchestrationWindow } from '../utils/taskOrchestrationWindowManager.js'

/**
 * Ribbon getImage 返回值（见 WPS 自定义功能区示例）：直接返回相对路径如 images/1.svg，
 * 由宿主按加载项根目录解析，无额外网络与 Canvas 开销。
 * 已为 http(s)/data:/file: 的绝对地址时原样返回（自定义助手图标等）。
 */
function resolveRibbonIconUrl(raw) {
  const s = String(raw ?? '').trim()
  if (!s) return s
  if (/^(https?:\/\/|data:|file:)/i.test(s)) return s
  return s.replace(/^\/+/, '')
}

//这个函数在整个wps加载项中是第一个执行的
function OnAddinLoad(ribbonUI) {
  if (typeof window.Application.ribbonUI != 'object') {
    window.Application.ribbonUI = ribbonUI
  }

  if (typeof window.Application.Enum != 'object') {
    // 如果没有内置枚举值
    window.Application.Enum = Util.WPS_Enum
  }

  //这几个导出函数是给外部业务系统调用的
  window.openOfficeFileFromSystemDemo = SystemDemo.openOfficeFileFromSystemDemo
  window.InvokeFromSystemDemo = SystemDemo.InvokeFromSystemDemo

  window.Application.PluginStorage.setItem('EnableFlag', false) //往PluginStorage中设置一个标记，用于控制两个按钮的置灰
  window.Application.PluginStorage.setItem('ApiEventFlag', false) //往PluginStorage中设置一个标记，用于控制ApiEvent的按钮label

  // 文档打开或切换时刷新表单模式按钮状态（从文档变量/保护类型读取）
  try {
    if (window.Application.ApiEvent) {
      window.Application.ApiEvent.AddApiEventListener('DocumentOpen', 'ribbon.OnDocumentOpenForFormMode')
      window.Application.ApiEvent.AddApiEventListener('WindowActivate', 'ribbon.OnWindowActivateForFormMode')
      window.Application.ApiEvent.AddApiEventListener('DocumentBeforeSave', 'ribbon.OnDocumentBeforeSave')
    }
  } catch (e) {
    console.warn('注册文档事件失败:', e)
  }

  // 先设置默认模型列表，确保有数据显示
  setDefaultModels()
  // 从存储恢复选中的模型索引；无有效选中时默认选中 OpenAI
  const storedIndex = window.Application.PluginStorage.getItem('selectedModelIndex')
  if (storedIndex !== undefined && storedIndex !== null) {
```

```
    const idx = parseInt(storedIndex, 10)
    if (!isNaN(idx)) selectedModelIndex = idx
  } else {
    selectedModelIndex = getOpenAIModelIndex(getModelList())
  }
  // 然后尝试从接口加载（异步）
  loadModelList()

  return true
}

function openAIAssistantDialog(query = {}) {
  const queryString = new URLSearchParams(query).toString()
  const aiUrl = Util.GetUrlPath() + Util.GetRouterHash() + `/ai-assistant${queryString ? `?${queryString}` : ''}`
  window.Application.ShowDialog(
    aiUrl,
    '察元 AI 助手',
    900 * (window.devicePixelRatio || 1),
    700 * (window.devicePixelRatio || 1),
    false
  )
}

function showAIAssistantDialog(query = {}) {
  if (reopenExistingAIAssistantWindow(query)) {
    openAIAssistantDialog({
      ...query,
      reopen: '1'
    })
    return
  }
  openAIAssistantDialog(query)
}


// 模型列表数据
let modelList = []
let selectedModelIndex = 0

// OpenAI 系列模型 id，用于默认选中
const OPENAI_MODEL_IDS = ['gpt-4o', 'gpt-4', 'gpt-4-turbo', 'o3', 'gpt_o1', 'gpt-5', 'gpt-3.5', 'openai']

// 获取列表中第一个 OpenAI 模型的索引，无则返回 0
function getOpenAIModelIndex(list) {
  if (!list || list.length === 0) return 0
  const idx = list.findIndex(m => m && OPENAI_MODEL_IDS.includes(String(m.id).toLowerCase()))
  return idx >= 0 ? idx : 0
}

// 从接口加载模型列表
```

```
async function loadModelList() {
  try {
    // TODO: 替换为实际的API接口地址
    // 暂时注释掉接口调用，直接使用默认模型列表进行测试
    // const apiUrl = 'https://your-api-endpoint.com/api/models' // 请替换为实际的接口地址
    const apiUrl = null // 暂时禁用接口调用，使用默认数据

    if (!apiUrl) {
      // 如果接口地址未配置，直接使用默认模型列表
      console.log('使用默认模型列表')
      return
    }

    // 如果接口需要认证，可以在这里添加headers
    const response = await fetch(apiUrl, {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
        // 'Authorization': 'Bearer your-token' // 如果需要认证
      }
    })

    if (response.ok) {
      const data = await response.json()
      // 假设接口返回格式为: { models: [{ id: '1', name: 'GPT-4', icon: '...' }, ...] }
      // 或者直接返回数组: [{ id: '1', name: 'GPT-4', icon: '...' }, ...]
      modelList = Array.isArray(data) ? data : (data.models || data.data || [])

      // 保存到本地存储
      window.Application.PluginStorage.setItem('modelList', JSON.stringify(modelList))

      // 刷新模型选择菜单
      if (window.Application.ribbonUI) {
        window.Application.ribbonUI.InvalidateControl('menuModelSelect')
      }
    } else {
      console.error('获取模型列表失败:', response.statusText)
      // 使用默认模型列表
      setDefaultModels()
    }
  } catch (error) {
    console.error('加载模型列表出错:', error)
    // 使用默认模型列表
    setDefaultModels()
  }
}

// 设置默认模型列表（使用全球主流模型列表）
function setDefaultModels() {
  modelList = getDefaultModelsFlat()
```

```
  window.Application.PluginStorage.setItem('modelList', JSON.stringify(modelList))
  // 刷新模型选择菜单
  if (window.Application.ribbonUI) {
    window.Application.ribbonUI.InvalidateControl('menuModelSelect')
  }
}

// 获取模型列表（从本地存储或默认值）
function getModelList() {
  if (modelList.length > 0) {
    return modelList
  }

  const stored = window.Application.PluginStorage.getItem('modelList')
  if (stored) {
    try {
      modelList = JSON.parse(stored)
      return modelList
    } catch (e) {
      console.error('解析模型列表失败:', e)
    }
  }

  setDefaultModels()
  return modelList
}

// 获取在模型设置中已开启的模型列表（仅显示在察元AI助理下拉中可选的模型）
function getFilteredModelList() {
  const full = getModelList()
  return full.filter(m => m && m.id && isRibbonModelEnabled(m.id))
}

// 获取当前选中的模型索引（在过滤后的列表中）
function getSelectedModelIndex() {
  const list = getFilteredModelList()
  const storedModel = window.Application?.PluginStorage?.getItem('selectedModel')
  if (storedModel) {
    try {
      const obj = JSON.parse(storedModel)
      const idx = list.findIndex(m => m && m.id === obj?.id)
      if (idx >= 0) return idx
    } catch (e) {}
  }
  const storedIdx = window.Application?.PluginStorage?.getItem('selectedModelIndex')
  if (storedIdx !== undefined && storedIdx !== null) {
    const idx = parseInt(storedIdx, 10)
    if (!isNaN(idx) && idx >= 0 && idx < list.length) return idx
  }
  return getOpenAIModelIndex(list)
```

```
    : response.data
  return {
    output,
    preview: `已请求 ${method} ${url}，状态 ${response.status}`
  }
}

async function executeJsonExtract(node, context) {
  const config = node?.data?.config || {}
  const rawInputValue = context?.inputValue
  const rawInput = String(context?.inputText || '').trim()
  if (rawInputValue == null && !rawInput) {
    throw new Error('JSON 提取节点没有收到上游数据')
  }
  const parsed = rawInputValue != null && typeof rawInputValue === 'object'
    ? rawInputValue
    : safeParseJson(rawInput, null)
  if (parsed == null) {
    throw new Error('JSON 提取节点收到的内容不是有效 JSON')
  }
  const extracted = getByPath(parsed, config.path)
  const output = extracted == null || extracted === ''
    ? String(config.fallbackValue || '')
    : extracted
  return {
    output,
    preview: String(config.path || 'root').trim()
      ? `已提取字段 ${String(config.path).trim()}`
      : '已输出完整 JSON'
  }
}

async function executeTextTemplate(node, context) {
  const config = node?.data?.config || {}
  const output = interpolateTemplate(config.template || '{{input}}', buildToolVariables(context))
  return {
    output,
    preview: '已按模板生成文本'
  }
}

async function executeFieldMapper(node, context) {
  const config = node?.data?.config || {}
  const fieldName = String(config.fieldName || 'payload').trim() || 'payload'
  const mapped = {
    [fieldName]: context?.inputValue != null ? context.inputValue : String(context?.inputText || '')
  }
  if (config.includeMeta !== false) {
    mapped._meta = {
      workflowName: String(context?.workflow?.name || '').trim(),
```

```
      nodeTitle: String(node?.data?.title || node?.label || '').trim(),
      parentCount: Array.isArray(context?.parentOutputs) ? context.parentOutputs.length : 0
    }
  }
  return {
    output: mapped,
    preview: `已映射为字段 ${fieldName}`
  }
}

async function executeContentMerge(node, context) {
  const config = node?.data?.config || {}
  const parentOutputs = Array.isArray(context?.parentOutputs) ? context.parentOutputs : []
  const parts = parentOutputs
    .map(item => String(item?.text || '').trim())
    .filter(Boolean)
  const separator = String(config.separator ?? '\n\n')
  const merged = parts.join(separator)
  if (config.template) {
    return {
      output: interpolateTemplate(config.template, {
        ...buildToolVariables(context),
        merged
      }),
      preview: `已合并 ${parts.length} 段内容`
    }
  }
  return {
    output: merged,
    preview: `已合并 ${parts.length} 段内容`
  }
}

async function executeTextReplace(node, context) {
  const config = node?.data?.config || {}
  const source = String(context?.inputText || '')
  if (!String(config.searchValue || '').trim()) {
    throw new Error('文本替换节点缺少查找内容')
  }
  let output = source
  if (config.useRegex) {
    const regex = new RegExp(String(config.searchValue || ''), String(config.regexFlags || 'g'))
    output = source.replace(regex, String(config.replaceValue || ''))
  } else {
    const searchValue = String(config.searchValue || '')
    const replaceValue = String(config.replaceValue || '')
    if (config.caseSensitive === false) {
      const escaped = searchValue.replace(/[.*+?^${}()|[]\\]/g, '\\$&')
      output = source.replace(new RegExp(escaped, 'gi'), replaceValue)
    } else {
```

```
      output = source.split(searchValue).join(replaceValue)
    }
  }
  return {
    output,
    preview: '已完成文本替换'
  }
}

async function executeRegexExtract(node, context) {
  const config = node?.data?.config || {}
  const source = String(context?.inputText || '')
  const pattern = String(config.pattern || '').trim()
  if (!pattern) {
    throw new Error('正则提取节点缺少表达式')
  }
  const rawFlags = String(config.flags || 'g')
  const regex = new RegExp(pattern, rawFlags.includes('g') ? rawFlags : `${rawFlags}g`)
  const matches = [...source.matchAll(regex)]
  if (matches.length === 0) {
    return {
      output: '',
      preview: '未匹配到内容'
    }
  }
  const groupIndex = Math.max(0, Number(config.groupIndex || 0))
  const values = matches.map(match => match?.[groupIndex] ?? match?.[0] ?? '').filter(Boolean)
  const matchIndex = Number(config.matchIndex ?? 0)
  const output = matchIndex === -1
    ? values.join(String(config.joinSeparator ?? '\n'))
    : (values[matchIndex] ?? '')
  return {
    output,
    preview: matchIndex === -1 ? `已提取 ${values.length} 条匹配` : '已提取目标匹配'
  }
}

async function executeConditionCheck(node, context) {
  const config = node?.data?.config || {}
  const rawInput = String(context?.inputText || '')
  const subjectPath = String(config.subjectPath || '').trim()
  let targetValue = context?.inputValue
  if (subjectPath) {
    const payload = {
      value: context?.inputValue,
      text: rawInput
    }
    const extracted = getValueFromPayload(payload, subjectPath)
    if (extracted !== undefined) targetValue = extracted
  }
```

```
  const subjectText = stringifyWorkflowValue(targetValue != null ? targetValue : rawInput)
  const input = config.trimInput === false ? subjectText : subjectText.trim()
  const expectedValue = String(config.expectedValue || '')
  const mode = String(config.mode || 'contains').trim()
  let matched = false
  if (mode === 'contains') {
    matched = expectedValue ? input.includes(expectedValue) : false
  } else if (mode === 'equals') {
    matched = input === expectedValue
  } else if (mode === 'not-empty') {
    matched = input.length > 0
  } else if (mode === 'regex') {
    matched = expectedValue ? new RegExp(expectedValue, String(config.regexFlags || 'i')).test(input) : false
  }
  const output = {
    matched,
    mode,
    input,
    subjectPath,
    expectedValue,
    resultText: matched ? String(config.trueValue || '命中') : String(config.falseValue || '未命中')
  }
  return {
    output,
    preview: matched ? '条件判断结果：命中' : '条件判断结果：未命中'
  }
}

/** 编排时调试：模拟条件判断结果（不执行，仅根据输入和配置计算） */
export function simulateConditionCheck(node, context) {
  const config = node?.data?.config || {}
  const rawInput = String(context?.inputText || '')
  const subjectPath = String(config.subjectPath || '').trim()
  let targetValue = context?.inputValue
  if (subjectPath) {
    const payload = { value: context?.inputValue, text: rawInput }
    const extracted = getValueFromPayload(payload, subjectPath)
    if (extracted !== undefined) targetValue = extracted
  }
  const subjectText = stringifyWorkflowValue(targetValue != null ? targetValue : rawInput)
  const input = config.trimInput === false ? subjectText : subjectText.trim()
  const expectedValue = String(config.expectedValue || '')
  const mode = String(config.mode || 'contains').trim()
  let matched = false
  if (mode === 'contains') {
    matched = expectedValue ? input.includes(expectedValue) : false
  } else if (mode === 'equals') {
    matched = input === expectedValue
  } else if (mode === 'not-empty') {
    matched = input.length > 0
```

```
  } else if (mode === 'regex') {
    matched = expectedValue ? new RegExp(expectedValue, String(config.regexFlags || 'i')).test(input) : false
  }
  return {
    matched,
    subjectText: input,
    resultText: matched ? String(config.trueValue || '命中') : String(config.falseValue || '未命中')
  }
}

/** 编排时调试：模拟 JSON 提取结果 */
export function simulateJsonExtract(node, context) {
  const config = node?.data?.config || {}
  const rawInputValue = context?.inputValue
  const rawInput = String(context?.inputText || '').trim()
  if (rawInputValue == null && !rawInput) return null
  const parsed = rawInputValue != null && typeof rawInputValue === 'object'
    ? rawInputValue
    : safeParseJson(rawInput, null)
  if (parsed == null) return null
  const extracted = getByPath(parsed, config.path)
  return extracted == null || extracted === ''
    ? String(config.fallbackValue || '')
    : extracted
}

async function executeDelay(node, context) {
  const config = node?.data?.config || {}
  const delayMs = Math.max(0, Number(config.delayMs || 0))
  await sleep(delayMs)
  return {
    output: String(context?.inputText || ''),
    preview: `已等待 ${delayMs} ms`
  }
}

async function executeSetVariables(node) {
  const config = node?.data?.config || {}
  const valueType = String(config.valueType || 'text').trim().toLowerCase()
  if (valueType === 'json') {
    const parsed = safeParseJson(config.valueText, null)
    if (parsed == null) {
      throw new Error('固定变量节点中的 JSON 配置无效')
    }
    return {
      output: parsed,
      preview: '已输出固定 JSON'
    }
  }
  return {
```

```
    output: String(config.valueText || ''),
    preview: '已输出固定文本'
  }
}

async function executeCapabilityBusTool(node, context) {
  const config = node?.data?.config || {}
  const capabilityKey = String(config.capabilityKey || '').trim()
  if (!capabilityKey) {
    throw new Error('能力总线节点缺少 capabilityKey')
  }
  const variables = buildToolVariables(context)
  let params = {}
  if (config.paramsText) {
    const rendered = interpolateTemplate(String(config.paramsText || '{}'), {
      ...variables,
      capabilityCatalog: getCapabilityCatalogSummary()
    })
    params = safeParseJson(rendered, {}) || {}
  }
  const result = executeCapabilityBusRequest({
    capabilityKey,
    params,
    inputText: context?.inputText || '',
    entry: 'workflow',
    launchSource: 'workflow',
    confirmed: true,
    workflowId: context?.workflow?.id || '',
    workflowName: context?.workflow?.name || ''
  })
  return {
    output: result,
    preview: `已执行 ${result.capabilityLabel || capabilityKey}`
  }
}

export async function executeWorkflowTool(node, context = {}) {
  const toolType = String(node?.data?.toolType || '').trim()
  switch (toolType) {
    case 'http-request':
      return executeHttpRequest(node, context)
    case 'json-extract':
      return executeJsonExtract(node, context)
    case 'text-template':
      return executeTextTemplate(node, context)
    case 'field-mapper':
      return executeFieldMapper(node, context)
    case 'content-merge':
      return executeContentMerge(node, context)
    case 'text-replace':
```

```
      return executeTextReplace(node, context)
    case 'regex-extract':
      return executeRegexExtract(node, context)
    case 'condition-check':
      return executeConditionCheck(node, context)
    case 'delay':
      return executeDelay(node, context)
    case 'set-variables':
      return executeSetVariables(node, context)
    case 'capability-bus':
    case 'wps-capability':
      return executeCapabilityBusTool(node, context)
    default:
      throw new Error(`暂不支持执行工具类型：${toolType || 'unknown'}`)
  }
}

export function getWorkflowToolOutputText(result) {
  return stringifyOutput(result?.output)
}

import { getCurrentDocumentSavePath } from './documentFileActions.js'
import { getSelectionContextSnapshot } from './documentContext.js'

function parseRequestedPageNumber(text = '') {
  const matched = String(text || '').match(/第\s*([0-9]{1,4})\s*页/)
  return matched ? Number(matched[1]) : 0
}

function parseTableShape(text = '') {
  const normalized = String(text || '')
  const matched = normalized.match(/([0-9]{1,3})\s*[xX * *]\s*([0-9]{1,3})/)
  if (matched) {
    return {
      rows: Math.max(1, Number(matched[1] || 0)),
      columns: Math.max(1, Number(matched[2] || 0))
    }
  }
  const rowsMatched = normalized.match(/([0-9]{1,3})\s*行/)
  const columnsMatched = normalized.match(/([0-9]{1,3})\s*列/)
  return {
    rows: Math.max(1, Number(rowsMatched?.[1] || 3)),
    columns: Math.max(1, Number(columnsMatched?.[1] || 3))
  }
}

function buildPageNumberDefault(text = '') {
  const parsed = parseRequestedPageNumber(text)
  if (parsed > 0) return parsed
  const snapshot = getSelectionContextSnapshot()
```

```
  const currentPage = Number(snapshot?.documentStats?.currentPage || 0)
  return currentPage > 0 ? currentPage : 1
}

function buildSavePathDefault() {
  return getCurrentDocumentSavePath() || ''
}

export const WPS_CAPABILITIES = [
  {
    capabilityKey: 'save-document',
    label: '保存文档',
    category: 'document-file',
    description: '保存当前已打开的文档。',
    routingPromptHint: '当用户明确要求保存当前文档、保存文件时使用。',
    executeMode: 'direct',
    executorKey: 'save-document',
    requiredParams: [],
    optionalParams: [
      {
        key: 'savePath',
        label: '保存路径',
        type: 'path',
        placeholder: '当前文档未保存时，可填写保存路径',
        required: false,
        defaultResolver: () => buildSavePathDefault()
      }
    ]
  },
  {
    capabilityKey: 'save-document-as',
    label: '文档另存为',
    category: 'document-file',
    description: '将当前文档另存为新的路径。',
    routingPromptHint: '当用户要求另存为、保存到某个位置、导出到指定路径时使用。',
    executeMode: 'direct',
    executorKey: 'save-document-as',
    requiredParams: [
      {
        key: 'savePath',
        label: '保存路径',
        type: 'path',
        placeholder: '请输入完整保存路径，例如 /Users/name/Desktop/文档.docx',
        required: true,
        defaultResolver: () => buildSavePathDefault()
      }
    ],
    optionalParams: []
  },
  {
```

```
  capabilityKey: 'save-document-with-dialog',
  label: '选择路径后另存',
  category: 'document-file',
  description: '先弹出保存位置选择，再将当前文档另存到指定路径。',
  routingPromptHint: '当用户希望弹出保存框、手动选择路径后再另存时使用。',
  executeMode: 'direct',
  executorKey: 'save-document-with-dialog',
  requiredParams: [],
  optionalParams: []
},
{
  capabilityKey: 'encrypt-document',
  label: '加密文档',
  category: 'document-security',
  description: '为当前文档设置密码并保存。',
  routingPromptHint: '当用户要求给文档加密、设置密码、加保护密码保存时使用。',
  executeMode: 'direct',
  executorKey: 'encrypt-document',
  requiredParams: [
    {
      key: 'password',
      label: '文档密码',
      type: 'password',
      placeholder: '请输入文档密码',
      required: true,
      defaultResolver: () => ''
    }
  ],
  optionalParams: [
    {
      key: 'savePath',
      label: '保存路径',
      type: 'path',
      placeholder: '如需另存为加密文档，可填写新路径',
      required: false,
      defaultResolver: () => buildSavePathDefault()
    }
  ]
},
{
  capabilityKey: 'encrypt-document-with-dialog',
  label: '选择路径后加密保存',
  category: 'document-security',
  description: '先选择保存位置，再为当前文档设置密码并保存。',
  routingPromptHint: '当用户希望选择路径后再加密保存、另存为加密文档时使用。',
  executeMode: 'direct',
  executorKey: 'encrypt-document-with-dialog',
  requiredParams: [
    {
      key: 'password',
```

```
      label: '文档密码',
      type: 'password',
      placeholder: '请输入文档密码',
      required: true,
      defaultResolver: () => ''
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'decrypt-document',
  label: '解密文档',
  category: 'document-security',
  description: '移除当前文档密码并重新保存。',
  routingPromptHint: '当用户要求解密文档、移除文档密码、取消加密保护时使用。',
  executeMode: 'direct',
  executorKey: 'decrypt-document',
  requiredParams: [],
  optionalParams: [
    {
      key: 'savePath',
      label: '保存路径',
      type: 'path',
      placeholder: '如需另存为解密文档，可填写新路径',
      required: false,
      defaultResolver: () => buildSavePathDefault()
    }
  ]
},
{
  capabilityKey: 'insert-table',
  label: '插入表格',
  category: 'document-structure',
  description: '在指定位置插入表格。',
  routingPromptHint: '当用户要求插入表格、创建几行几列表格时使用。',
  executeMode: 'direct',
  executorKey: 'insert-table',
  requiredParams: [
    {
      key: 'rows',
      label: '表格行数',
      type: 'number',
      placeholder: '请输入行数',
      required: true,
      defaultResolver: (text) => parseTableShape(text).rows
    },
    {
      key: 'columns',
      label: '表格列数',
      type: 'number',
```

```
      placeholder: '请输入列数',
      required: true,
      defaultResolver: (text) => parseTableShape(text).columns
    },
    {
      key: 'pageNumber',
      label: '插入页码',
      type: 'page-selector',
      placeholder: '请输入页码',
      required: true,
      defaultResolver: (text) => buildPageNumberDefault(text)
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'insert-page-break',
  label: '插入分页符',
  category: 'document-structure',
  description: '在指定位置插入分页符。',
  routingPromptHint: '当用户要求插入分页符、分页时使用。',
  executeMode: 'direct',
  executorKey: 'insert-page-break',
  requiredParams: [
    {
      key: 'pageNumber',
      label: '插入页码',
      type: 'page-selector',
      placeholder: '请输入页码',
      required: true,
      defaultResolver: (text) => buildPageNumberDefault(text)
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'insert-blank-page',
  label: '插入空白页',
  category: 'document-structure',
  description: '在指定位置插入空白页。',
  routingPromptHint: '当用户要求插入空白页、新增空白页时使用。',
  executeMode: 'direct',
  executorKey: 'insert-blank-page',
  requiredParams: [
    {
      key: 'pageNumber',
      label: '插入页码',
      type: 'page-selector',
      placeholder: '请输入页码',
      required: true,
```

```
      defaultResolver: (text) => buildPageNumberDefault(text)
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'replace-selection-text',
  label: '替换当前选中内容',
  category: 'document-edit',
  description: '用新文本替换当前选中内容。',
  routingPromptHint: '当用户要求把当前选中文字替换成指定内容时使用。',
  executeMode: 'direct',
  executorKey: 'replace-selection-text',
  requiredParams: [
    {
      key: 'text',
      label: '替换内容',
      type: 'textarea',
      placeholder: '请输入替换后的文本',
      required: true,
      defaultResolver: () => ''
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'paste-text',
  label: '插入文本',
  category: 'document-edit',
  description: '在当前光标或选区位置插入文本。',
  routingPromptHint: '当用户要求粘贴、插入一段文字到当前位置时使用。',
  executeMode: 'direct',
  executorKey: 'paste-text',
  requiredParams: [
    {
      key: 'text',
      label: '插入内容',
      type: 'textarea',
      placeholder: '请输入要插入的文本',
      required: true,
      defaultResolver: () => ''
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'append-text-to-document',
  label: '追加文本到文末',
  category: 'document-edit',
  description: '将文本追加到当前文档末尾。',
```

```
  routingPromptHint: '当用户要求把文本追加到文末、文档最后时使用。',
  executeMode: 'direct',
  executorKey: 'append-text-to-document',
  requiredParams: [
    {
      key: 'text',
      label: '追加内容',
      type: 'textarea',
      placeholder: '请输入要追加的文本',
      required: true,
      defaultResolver: () => ''
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'copy-current-paragraph',
  label: '复制当前段落',
  category: 'document-edit',
  description: '把当前段落复制到指定段落前后。',
  routingPromptHint: '当用户要求复制当前段落到某个段落前后时使用。',
  executeMode: 'direct',
  executorKey: 'copy-current-paragraph',
  requiredParams: [
    {
      key: 'destinationIndex',
      label: '目标段落序号',
      type: 'number',
      placeholder: '请输入目标段落序号',
      required: true,
      defaultResolver: () => 1
    }
  ],
  optionalParams: [
    {
      key: 'placement',
      label: '插入位置',
      type: 'select',
      options: ['before', 'after'],
      placeholder: 'before 或 after',
      required: false,
      defaultResolver: () => 'after'
    }
  ]
},
{
  capabilityKey: 'duplicate-selection-text',
  label: '复制当前选中内容并插入',
  category: 'document-edit',
  description: '读取当前选中文本，并在当前位置再次插入一份。',
```

```
  routingPromptHint: '当用户要求复制当前选中内容、把这段话再插入一份时使用。',
  executeMode: 'direct',
  executorKey: 'duplicate-selection-text',
  requiredParams: [],
  optionalParams: []
},
{
  capabilityKey: 'set-font-name',
  label: '设置字体',
  category: 'document-format',
  description: '设置当前选区字体。',
  routingPromptHint: '当用户要求把选中文字改成某种字体时使用。',
  executeMode: 'direct',
  executorKey: 'set-font-name',
  requiredParams: [
    {
      key: 'fontName',
      label: '字体名称',
      type: 'text',
      placeholder: '如 宋体、微软雅黑',
      required: true,
      defaultResolver: () => ''
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'set-font-size',
  label: '设置字号',
  category: 'document-format',
  description: '设置当前选区字号。',
  routingPromptHint: '当用户要求把选中文字调成某个字号时使用。',
  executeMode: 'direct',
  executorKey: 'set-font-size',
  requiredParams: [
    {
      key: 'fontSize',
      label: '字号',
      type: 'number',
      placeholder: '如 12',
      required: true,
      defaultResolver: () => 12
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'set-font-color',
  label: '设置文字颜色',
  category: 'document-format',
```

```
  description: '设置当前选区文字颜色。',
  routingPromptHint: '当用户要求把选中文字改成某种颜色时使用。',
  executeMode: 'direct',
  executorKey: 'set-font-color',
  requiredParams: [
    {
      key: 'fontColor',
      label: '文字颜色',
      type: 'text',
      placeholder: '如 红色、#FF0000',
      required: true,
      defaultResolver: () => ''
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'set-background-color',
  label: '设置背景色',
  category: 'document-format',
  description: '设置当前选区背景色或高亮色。',
  routingPromptHint: '当用户要求把选中文字背景改色、高亮时使用。',
  executeMode: 'direct',
  executorKey: 'set-background-color',
  requiredParams: [
    {
      key: 'backgroundColor',
      label: '背景颜色',
      type: 'text',
      placeholder: '如 黄色、#FFFF00',
      required: true,
      defaultResolver: () => ''
    }
  ],
  optionalParams: []
},
{
  capabilityKey: 'toggle-bold',
  label: '设置加粗',
  category: 'document-format',
  description: '把当前选区设置为加粗或取消加粗。',
  routingPromptHint: '当用户要求加粗、取消加粗时使用。',
  executeMode: 'direct',
  executorKey: 'toggle-bold',
  requiredParams: [],
  optionalParams: [
    {
      key: 'enabled',
      label: '是否加粗',
      type: 'boolean',
```

```
      placeholder: 'true / false',
      required: false,
      defaultResolver: () => true
    }
  ]
},
{
  capabilityKey: 'toggle-italic',
  label: '设置斜体',
  category: 'document-format',
  description: '把当前选区设置为斜体或取消斜体。',
  routingPromptHint: '当用户要求斜体、取消斜体时使用。',
  executeMode: 'direct',
  executorKey: 'toggle-italic',
  requiredParams: [],
  optionalParams: [
    {
      key: 'enabled',
      label: '是否斜体',
      type: 'boolean',
      placeholder: 'true / false',
      required: false,
      defaultResolver: () => true
    }
  ]
},
{
  capabilityKey: 'toggle-underline',
  label: '设置下划线',
  category: 'document-format',
  description: '把当前选区设置为下划线或取消下划线。',
  routingPromptHint: '当用户要求添加下划线、取消下划线时使用。',
  executeMode: 'direct',
  executorKey: 'toggle-underline',
  requiredParams: [],
  optionalParams: [
    {
      key: 'enabled',
      label: '是否加下划线',
      type: 'boolean',
      placeholder: 'true / false',
      required: false,
      defaultResolver: () => true
    }
  ]
},
{
  capabilityKey: 'set-alignment',
  label: '设置对齐方式',
  category: 'document-format',
```

```
    description: '设置当前选区段落对齐方式。',
    routingPromptHint: '当用户要求左对齐、居中、右对齐、两端对齐时使用。',
    executeMode: 'direct',
    executorKey: 'set-alignment',
    requiredParams: [
      {
        key: 'alignment',
        label: '对齐方式',
        type: 'select',
        options: ['left', 'center', 'right', 'justify'],
        placeholder: 'left / center / right / justify',
        required: true,
        defaultResolver: () => 'left'
      }
    ],
    optionalParams: []
  },
  {
    capabilityKey: 'set-line-spacing',
    label: '设置行距',
    category: 'document-format',
    description: '设置当前选区段落行距。',
    routingPromptHint: '当用户要求设置 1.5 倍行距、双倍行距等时使用。',
    executeMode: 'direct',
    executorKey: 'set-line-spacing',
    requiredParams: [
      {
        key: 'lineSpacing',
        label: '行距',
        type: 'text',
        placeholder: '如 1.5、double',
        required: true,
        defaultResolver: () => '1.5'
      }
    ],
    optionalParams: []
  }
]

export function getWpsCapabilityCatalog() {
  return WPS_CAPABILITIES.map(item => ({
    ...item,
    requiredParams: (item.requiredParams || []).map(param => ({ ...param })),
    optionalParams: (item.optionalParams || []).map(param => ({ ...param }))
  }))
}

export function getWpsCapabilityByKey(key) {
  const normalized = String(key || '').trim()
  return getWpsCapabilityCatalog().find(item => item.capabilityKey === normalized) || null
```

```
}

import { addTask, updateTask } from './taskListStore.js'
import { getWpsCapabilityByKey } from './wpsCapabilityCatalog.js'
import { openDocumentSaveAsDialog, saveActiveDocument, saveActiveDocumentAs } from './documentFileActions.js'
import { decryptActiveDocument, encryptActiveDocument } from './documentSecurityActions.js'
import { insertBlankPageAtPosition, insertPageBreakAtPosition, insertTableAtPosition } from './documentInsertActions.js'
import { applyDocumentAction, getSelectedText } from './documentActions.js'
import { executeDocumentFormatAction } from './documentFormatActions.js'
import { executeDocumentRelocationAction } from './documentRelocationActions.js'
import { buildGeneratedArtifactDescriptor, mergeTaskOrchestrationData } from './taskOrchestrationMeta.js'
import { appendCapabilityAuditRecord } from './capabilityAuditStore.js'
import { evaluateCapabilityPolicy, inferCapabilityRiskLevel } from './capabilityPolicyStore.js'
import { appendCapabilityQuotaUsage, evaluateCapabilityQuota } from './capabilityQuotaStore.js'

const activeCapabilityRuns = new Map()

function createCancelError() {
  const err = new Error('任务已停止')
  err.code = 'TASK_CANCELLED'
  return err
}

function throwIfCancelled(runState) {
  if (runState?.cancelled) throw createCancelError()
}

function getPreview(text, limit = 200) {
  const value = String(text || '').replace(/\s+/g, ' ').trim()
  if (!value) return ''
  return value.length > limit ? `${value.slice(0, limit)}...` : value
}

function appendEvent(events = [], text = '') {
  const value = String(text || '').trim()
  if (!value) return Array.isArray(events) ? [...events] : []
  return [...(Array.isArray(events) ? events : []), value].slice(-50)
}

function createItem(index, title, content, status = 'done', extra = {}) {
  return {
    index,
    title,
    chunkIndex: index,
    chunkText: String(content || '').trim(),
    status,
    output: String(extra.output || '').trim(),
    issues: Array.isArray(extra.issues) ? extra.issues : []
  }
}
```

```
function buildCapabilityWriteTargets(capabilityKey, params, result) {
  const safeResult = result && typeof result === 'object' ? result : {}
  if (['save-document', 'save-document-as', 'save-document-with-dialog', 'encrypt-document', 'encrypt-document-with-dialog', 'decrypt-document'].includes(capa…
    return []
  }
  if (capabilityKey === 'insert-table') {
    return [{
      action: 'insert-table',
      start: null,
      end: null,
      paragraphIndex: Number.isFinite(Number(params?.pageNumber)) ? Number(params.pageNumber) : null,
      originalText: '',
      outputText: `${Number(safeResult.rows || params?.rows || 0)}x${Number(safeResult.columns || params?.columns || 0)} 表格`,
      downgraded: false,
      locateKey: `wps_insert_table_${Number(params?.pageNumber || 0)}`
    }]
  }
  if (capabilityKey === 'insert-page-break' || capabilityKey === 'insert-blank-page') {
    return [{
      action: capabilityKey,
      start: null,
      end: null,
      paragraphIndex: Number.isFinite(Number(params?.pageNumber)) ? Number(params.pageNumber) : null,
      originalText: '',
      outputText: capabilityKey === 'insert-page-break' ? '分页符' : '空白页',
      downgraded: false,
      locateKey: `wps_${capabilityKey}_${Number(params?.pageNumber || 0)}`
    }]
  }
  if (['replace-selection-text', 'paste-text', 'append-text-to-document', 'duplicate-selection-text'].includes(capabilityKey)) {
    return [{
      action: capabilityKey,
      start: null,
      end: null,
      paragraphIndex: null,
      originalText: '',
      outputText: capabilityKey === 'duplicate-selection-text'
        ? String(getSelectedText() || '').trim()
        : String(params?.text || '').trim(),
      downgraded: false,
      locateKey: `wps_${capabilityKey}`
    }]
  }
  if ([
    'set-font-name',
    'set-font-size',
    'set-font-color',
    'set-background-color',
    'toggle-bold',
```

```
    'toggle-italic',
    'toggle-underline',
    'set-alignment',
    'set-line-spacing'
  ].includes(capabilityKey)) {
    const replayPayload = {
      scope: 'selection',
      fontName: params.fontName,
      fontSize: params.fontSize,
      fontColor: params.fontColor || params.color,
      backgroundColor: params.backgroundColor || params.color,
      bold: capabilityKey === 'toggle-bold' ? params.enabled !== false : undefined,
      italic: capabilityKey === 'toggle-italic' ? params.enabled !== false : undefined,
      underline: capabilityKey === 'toggle-underline' ? params.enabled !== false : undefined,
      alignment: capabilityKey === 'set-alignment' ? params.alignment || 'left' : undefined,
      lineSpacing: capabilityKey === 'set-line-spacing' ? params.lineSpacing || 1.5 : undefined
    }
    return [{
      action: 'format-selection',
      start: null,
      end: null,
      paragraphIndex: null,
      originalText: '',
      outputText: safeResult?.message || capabilityKey,
      downgraded: false,
      locateKey: `wps_${capabilityKey}`,
      replayPayload
    }]
  }
  return []
}

export function executeWpsCapabilityDirect(capabilityKey, params = {}) {
  if (capabilityKey === 'save-document') return saveActiveDocument(params)
  if (capabilityKey === 'save-document-as') return saveActiveDocumentAs(params.savePath)
  if (capabilityKey === 'save-document-with-dialog') {
    const savePath = openDocumentSaveAsDialog({ title: '另存文档' })
    if (!savePath) throw new Error('用户取消了另存为')
    return saveActiveDocumentAs(savePath)
  }
  if (capabilityKey === 'encrypt-document') return encryptActiveDocument(params)
  if (capabilityKey === 'encrypt-document-with-dialog') {
    const savePath = openDocumentSaveAsDialog({ title: '加密保存文档' })
    if (!savePath) throw new Error('用户取消了加密保存')
    return encryptActiveDocument({ ...params, savePath })
  }
  if (capabilityKey === 'decrypt-document') return decryptActiveDocument(params)
  if (capabilityKey === 'insert-table') return insertTableAtPosition(params)
  if (capabilityKey === 'insert-page-break') return insertPageBreakAtPosition(params)
  if (capabilityKey === 'insert-blank-page') return insertBlankPageAtPosition(params)
```

```
if (capabilityKey === 'replace-selection-text') {
  return applyDocumentAction('replace', String(params.text || '').trim(), params)
}
if (capabilityKey === 'paste-text') {
  return applyDocumentAction('insert', String(params.text || '').trim(), params)
}
if (capabilityKey === 'append-text-to-document') {
  return applyDocumentAction('append', String(params.text || '').trim(), params)
}
if (capabilityKey === 'copy-current-paragraph') {
  return executeDocumentRelocationAction({
    intent: 'copy',
    operation: 'copy',
    sourceType: 'current-paragraph',
    destinationType: 'paragraph-index',
    destinationIndex: Number(params.destinationIndex || 1),
    placement: String(params.placement || 'after').trim() || 'after',
    preserveFormatting: params.preserveFormatting !== false
  })
}
if (capabilityKey === 'duplicate-selection-text') {
  const selectedText = String(getSelectedText() || '').trim()
  if (!selectedText) {
    throw new Error('当前没有可复制的选中文本')
  }
  return applyDocumentAction('insert', selectedText, params)
}
if (capabilityKey === 'set-font-name') {
  return executeDocumentFormatAction({ scope: 'selection', fontName: params.fontName })
}
if (capabilityKey === 'set-font-size') {
  return executeDocumentFormatAction({ scope: 'selection', fontSize: Number(params.fontSize || 12) })
}
if (capabilityKey === 'set-font-color') {
  return executeDocumentFormatAction({ scope: 'selection', fontColor: params.fontColor || params.color })
}
if (capabilityKey === 'set-background-color') {
  return executeDocumentFormatAction({ scope: 'selection', backgroundColor: params.backgroundColor || params.color })
}
if (capabilityKey === 'toggle-bold') {
  return executeDocumentFormatAction({ scope: 'selection', bold: params.enabled !== false })
}
if (capabilityKey === 'toggle-italic') {
  return executeDocumentFormatAction({ scope: 'selection', italic: params.enabled !== false })
}
if (capabilityKey === 'toggle-underline') {
  return executeDocumentFormatAction({ scope: 'selection', underline: params.enabled !== false })
}
if (capabilityKey === 'set-alignment') {
  return executeDocumentFormatAction({ scope: 'selection', alignment: params.alignment || 'left' })
```

```
  }
  if (capabilityKey === 'set-line-spacing') {
    return executeDocumentFormatAction({ scope: 'selection', lineSpacing: params.lineSpacing || 1.5 })
  }
  throw new Error(`暂未实现能力：${capabilityKey}`)
}

function buildSuccessMessage(capabilityKey, result) {
  if (capabilityKey === 'save-document') {
    return `已保存文档：${result.fileName || result.path || '当前文档'}`
  }
  if (capabilityKey === 'save-document-as') {
    return `已将文档另存为：${result.fileName || result.path || '新文档'}`
  }
  if (capabilityKey === 'save-document-with-dialog') {
    return `已选择路径并另存文档：${result.fileName || result.path || '新文档'}`
  }
  if (capabilityKey === 'encrypt-document') {
    return `已加密并保存文档：${result.fileName || result.path || '当前文档'}`
  }
  if (capabilityKey === 'encrypt-document-with-dialog') {
    return `已选择路径并加密保存文档：${result.fileName || result.path || '当前文档'}`
  }
  if (capabilityKey === 'decrypt-document') {
    return `已移除文档密码并保存：${result.fileName || result.path || '当前文档'}`
  }
  if (capabilityKey === 'insert-table') {
    return `已插入 ${Number(result.rows || 0)} 行 ${Number(result.columns || 0)} 列表格`
  }
  if (capabilityKey === 'insert-page-break') {
    return `已插入分页符${Number(result.pageNumber || 0) > 0 ? `（第 ${Number(result.pageNumber)} 页）` : ''}`
  }
  if (capabilityKey === 'insert-blank-page') {
    return `已插入空白页${Number(result.pageNumber || 0) > 0 ? `（第 ${Number(result.pageNumber)} 页）` : ''}`
  }
  return result?.message || '操作已完成'
}

export function startWpsCapabilityTask(options = {}) {
  let taskId = ''
  const promise = executeWpsCapabilityTask({
    ...options,
    onTaskCreated: (id) => {
      taskId = id
      options.onTaskCreated?.(id)
    }
  })
  return { taskId, promise }
}
```

```
export async function executeWpsCapabilityTask(options = {}) {
  const capabilityKey = String(options.capabilityKey || '').trim()
  const capability = getWpsCapabilityByKey(capabilityKey)
  if (!capability) throw new Error('未找到对应的 WPS 能力')
  const requirementText = String(options.requirementText || '').trim()
  const params = options.params && typeof options.params === 'object' ? options.params : {}
  const riskLevel = inferCapabilityRiskLevel(capability, 'wps')
  const policyDecision = evaluateCapabilityPolicy({
    namespace: 'wps',
    capabilityKey,
    entry: 'wps-capability',
    launchSource: options.launchSource || 'dialog',
    confirmed: options.confirmed !== false
  }, capability)
  if (!policyDecision.allowed) {
    appendCapabilityAuditRecord({
      namespace: 'wps',
      capabilityKey,
      capabilityLabel: capability.label,
      status: policyDecision.decision === 'deny' ? 'denied' : 'failed',
      riskLevel,
      decision: policyDecision.decision,
      decisionReason: policyDecision.reason,
      confirmed: policyDecision.confirmed,
      entry: 'wps-capability',
      launchSource: options.launchSource || 'dialog',
      requirementText,
      params
    })
    throw new Error(policyDecision.reason || '能力策略拒绝执行')
  }
  const quotaDecision = evaluateCapabilityQuota({
    namespace: 'wps',
    capabilityKey,
    policySnapshot: policyDecision.policySnapshot
  })
  if (!quotaDecision.allowed) {
    appendCapabilityAuditRecord({
      namespace: 'wps',
      capabilityKey,
      capabilityLabel: capability.label,
      status: 'failed',
      riskLevel,
      decision: quotaDecision.decision,
      decisionReason: quotaDecision.reason,
      confirmed: policyDecision.confirmed,
      entry: 'wps-capability',
      launchSource: options.launchSource || 'dialog',
      requirementText,
      params
```

```
  })
  throw new Error(quotaDecision.reason || '能力调用已达到配额限制')
}
const taskTitle = String(options.taskTitle || capability.label || 'WPS 任务').trim() || 'WPS 任务'
const initialEvents = ['已创建 WPS 直接操作任务。', '正在校验参数并准备执行。']
const taskId = addTask({
  type: 'wps-capability',
  title: taskTitle,
  status: 'running',
  total: 3,
  current: 0,
  progress: 8,
  data: {
    capabilityKey,
    capabilityLabel: capability.label,
    inputPreview: getPreview(requirementText, 220),
    ...mergeTaskOrchestrationData({}, {
      entry: 'wps-capability',
      primaryIntent: 'wps-capability',
      executionMode: 'wps-task',
      launchSource: 'dialog'
    }, {
      progressStage: 'preparing'
    }),
    progressEvents: initialEvents,
    items: [],
    params,
    resultSummary: ''
  }
})
options.onTaskCreated?.(taskId)
const runState = {
  taskId,
  cancelled: false,
  startedAtMs: Date.now()
}
activeCapabilityRuns.set(taskId, runState)
try {
  const step1Events = appendEvent(initialEvents, '参数校验完成，开始调用 WPS API。')
  updateTask(taskId, {
    current: 1,
    progress: 30,
    data: mergeTaskOrchestrationData({}, {
      entry: 'wps-capability',
      primaryIntent: 'wps-capability',
      executionMode: 'wps-task',
      launchSource: 'dialog'
    }, {
      capabilityKey,
      capabilityLabel: capability.label,
```

```
    inputPreview: getPreview(requirementText, 220),
    progressStage: 'executing',
    progressEvents: step1Events,
    items: [createItem(1, '参数校验', JSON.stringify(params), 'done')],
    params
  })
})
throwIfCancelled(runState)

const result = executeWpsCapabilityDirect(capabilityKey, params)
appendCapabilityQuotaUsage({
  namespace: 'wps',
  capabilityKey
})
const elapsedMs = Date.now() - runState.startedAtMs
appendCapabilityAuditRecord({
  namespace: 'wps',
  capabilityKey,
  capabilityLabel: capability.label,
  status: 'completed',
  riskLevel,
  decision: policyDecision.decision,
  decisionReason: policyDecision.reason,
  confirmed: policyDecision.confirmed,
  entry: 'wps-capability',
  launchSource: 'dialog',
  taskId,
  requirementText,
  params,
  result,
  durationMs: elapsedMs
})
const successMessage = buildSuccessMessage(capabilityKey, result)
const finalEvents = appendEvent(step1Events, successMessage)
const generatedArtifacts = ['save-document', 'save-document-as', 'save-document-with-dialog', 'encrypt-document', 'encrypt-document-with-dialog'].includes…
  ? [buildGeneratedArtifactDescriptor({
    kind: capabilityKey === 'encrypt-document' || capabilityKey === 'encrypt-document-with-dialog' ? 'encrypted-document' : 'document',
    name: result?.fileName || '',
    path: result?.path || params?.savePath || '',
    status: 'ready'
  })].filter(item => item.path || item.name)
  : []
const writeTargets = buildCapabilityWriteTargets(capabilityKey, params, result)
updateTask(taskId, {
  status: 'completed',
  current: 3,
  progress: 100,
  data: mergeTaskOrchestrationData({}, {
    entry: 'wps-capability',
    primaryIntent: 'wps-capability',
```

```
      executionMode: 'wps-task',
      launchSource: 'dialog'
    }, {
      capabilityKey,
      capabilityLabel: capability.label,
      inputPreview: getPreview(requirementText, 220),
      outputPreview: successMessage,
      fullOutput: successMessage,
      resultSummary: successMessage,
      progressStage: 'completed',
      progressEvents: finalEvents,
      generatedArtifacts,
      writeTargets,
      estimatedRemainingMs: 0,
      elapsedMs,
      avgStepMs: Math.round(elapsedMs / 3),
      stepCount: 3,
      currentStepIndex: 3,
      items: [
        createItem(1, '参数校验', JSON.stringify(params), 'done'),
        createItem(2, '执行能力', capability.label, 'done', { output: JSON.stringify(result) }),
        createItem(3, '执行结果', successMessage, 'done')
      ],
      params,
      applyResult: {
        ok: true,
        action: capabilityKey,
        message: successMessage,
        result,
        writeTargets
      }
    })
  })
  return {
    taskId,
    result,
    message: successMessage
  }
} catch (error) {
  const failedText = error?.message || String(error)
  appendCapabilityAuditRecord({
    namespace: 'wps',
    capabilityKey,
    capabilityLabel: capability.label,
    status: error?.code === 'TASK_CANCELLED' ? 'cancelled' : 'failed',
    riskLevel,
    decision: policyDecision.decision,
    decisionReason: failedText || policyDecision.reason,
    confirmed: policyDecision.confirmed,
    entry: 'wps-capability',
```

```
      launchSource: 'dialog',
      taskId,
      requirementText,
      params,
      errorMessage: failedText,
      durationMs: Date.now() - runState.startedAtMs
    })
    updateTask(taskId, {
      status: error?.code === 'TASK_CANCELLED' ? 'cancelled' : 'failed',
      current: 3,
      progress: 100,
      error: failedText,
      data: mergeTaskOrchestrationData({}, {
        entry: 'wps-capability',
        primaryIntent: 'wps-capability',
        executionMode: 'wps-task',
        launchSource: 'dialog'
      }, {
        capabilityKey,
        capabilityLabel: capability.label,
        inputPreview: getPreview(requirementText, 220),
        outputPreview: failedText,
        fullOutput: failedText,
        resultSummary: failedText,
        progressStage: error?.code === 'TASK_CANCELLED' ? 'cancelled' : 'failed',
        progressEvents: appendEvent(['已创建 WPS 直接操作任务。'], failedText),
        estimatedRemainingMs: 0,
        elapsedMs: Date.now() - runState.startedAtMs,
        avgStepMs: 0,
        stepCount: 3,
        currentStepIndex: 3,
        items: [createItem(1, '执行失败', failedText, 'failed', { output: failedText })],
        params
      })
    })
    throw error
  } finally {
    activeCapabilityRuns.delete(taskId)
  }
}

export function stopWpsCapabilityTask(taskId) {
  const runState = activeCapabilityRuns.get(String(taskId || ''))
  if (!runState) return false
  runState.cancelled = true
  updateTask(taskId, {
    status: 'cancelled',
    error: '任务已停止'
  })
  return true
```

```
}

import { chatCompletion } from './chatApi.js'
import { getWpsCapabilityCatalog, getWpsCapabilityByKey } from './wpsCapabilityCatalog.js'

function parseJsonCandidate(raw) {
  const text = String(raw || '').trim()
  if (!text) return null
  const block = text.match(/```(?:json)?\s*([\s\S]*?)```/i)
  const candidate = block?.[1] ? block[1].trim() : text
  try {
    return JSON.parse(candidate)
  } catch (_) {
    const start = candidate.indexOf('{')
    const end = candidate.lastIndexOf('}')
    if (start >= 0 && end > start) {
      try {
        return JSON.parse(candidate.slice(start, end + 1))
      } catch (_) {
        return null
      }
    }
    return null
  }
}

export function routeWpsCapabilityByRule(text = '') {
  const normalized = String(text || '').trim()
  if (!normalized) return null
  if (/(另存为|另存文档|保存到|导出到).{0,24}(文档|文件)?/.test(normalized)) {
    return { capabilityKey: 'save-document-as', confidence: 'high', reason: '命中了另存为/保存到路径的直接操作表述。' }
  }
  if (/(弹出|打开|选择).{0,12}(保存框|保存窗口|保存路径|另存为窗口)/.test(normalized)) {
    return { capabilityKey: 'save-document-with-dialog', confidence: 'high', reason: '命中了先选择保存路径再执行文档保存的直接操作表述。' }
  }
  if (/(保存文档|保存文件|保存当前文档|保存一下文档|保存一下文件)/.test(normalized)) {
    return { capabilityKey: 'save-document', confidence: 'high', reason: '命中了保存文档的直接操作表述。' }
  }
  if (/(加密文档|给文档加密|设置文档密码|给文档设置密码)/.test(normalized)) {
    return { capabilityKey: 'encrypt-document', confidence: 'high', reason: '命中了文档加密/设置密码的直接操作表述。' }
  }
  if (/(解密文档|移除文档密码|取消文档密码|去掉文档密码|取消加密保护)/.test(normalized)) {
    return { capabilityKey: 'decrypt-document', confidence: 'high', reason: '命中了文档解密/移除密码的直接操作表述。' }
  }
  if (/(选择路径|弹出保存框|另存为).{0,12}(加密|密码保护)/.test(normalized)) {
    return { capabilityKey: 'encrypt-document-with-dialog', confidence: 'high', reason: '命中了先选择保存路径再加密保存的直接操作表述。' }
  }
  if (/(插入|新增|加一个).{0,12}(表格)/.test(normalized)) {
    return { capabilityKey: 'insert-table', confidence: 'high', reason: '命中了插入表格的直接操作表述。' }
  }
```

```
  if (/(插入|新增|加一个).{0,8}(分页符)/.test(normalized)) {
    return { capabilityKey: 'insert-page-break', confidence: 'high', reason: '命中了插入分页符的直接操作表述。' }
  }
  if (/(插入|新增|加一个).{0,8}(空白页|新页)/.test(normalized)) {
    return { capabilityKey: 'insert-blank-page', confidence: 'high', reason: '命中了插入空白页的直接操作表述。' }
  }
  if (/(替换|改成|修改成).{0,12}(选中|当前选中|这段文字)/.test(normalized)) {
    return { capabilityKey: 'replace-selection-text', confidence: 'high', reason: '命中了替换当前选中内容的直接操作表述。' }
  }
  if (/(插入|粘贴|追加).{0,12}(文字|文本|内容)/.test(normalized)) {
    return { capabilityKey: /(文末|最后)/.test(normalized) ? 'append-text-to-document' : 'paste-text', confidence: 'high', reason: '命中了插入或追加文本的直接操作表述。' }
  }
  if (/(复制).{0,12}(当前段落|这段|本段)/.test(normalized)) {
    return { capabilityKey: 'copy-current-paragraph', confidence: 'high', reason: '命中了复制当前段落的直接操作表述。' }
  }
  if (/(复制|拷贝|再插入一份|重复插入).{0,12}(当前选中|选中文本|这段文字)/.test(normalized)) {
    return { capabilityKey: 'duplicate-selection-text', confidence: 'high', reason: '命中了复制当前选中文本并插入的直接操作表述。' }
  }
  if (/(字体).{0,8}(改为|设为)/.test(normalized)) {
    return { capabilityKey: 'set-font-name', confidence: 'high', reason: '命中了设置字体的直接操作表述。' }
  }
  if (/(字号|字体大小).{0,8}(改为|设为)|改成.{0,6}号字/.test(normalized)) {
    return { capabilityKey: 'set-font-size', confidence: 'high', reason: '命中了设置字号的直接操作表述。' }
  }
  if (/(文字颜色|字色|字体颜色|颜色).{0,8}(改为|设为)/.test(normalized)) {
    return { capabilityKey: 'set-font-color', confidence: 'high', reason: '命中了设置文字颜色的直接操作表述。' }
  }
  if (/(背景色|高亮|底色).{0,8}(改为|设为|标成)/.test(normalized)) {
    return { capabilityKey: 'set-background-color', confidence: 'high', reason: '命中了设置背景色或高亮的直接操作表述。' }
  }
  if (/(加粗)/.test(normalized)) {
    return { capabilityKey: 'toggle-bold', confidence: 'high', reason: '命中了加粗的直接操作表述。' }
  }
  if (/(斜体)/.test(normalized)) {
    return { capabilityKey: 'toggle-italic', confidence: 'high', reason: '命中了斜体的直接操作表述。' }
  }
  if (/(下划线)/.test(normalized)) {
    return { capabilityKey: 'toggle-underline', confidence: 'high', reason: '命中了下划线的直接操作表述。' }
  }
  if (/(左对齐|右对齐|居中|两端对齐)/.test(normalized)) {
    return { capabilityKey: 'set-alignment', confidence: 'high', reason: '命中了段落对齐的直接操作表述。' }
  }
  if (/(行距)/.test(normalized)) {
    return { capabilityKey: 'set-line-spacing', confidence: 'high', reason: '命中了设置行距的直接操作表述。' }
  }
  return null
}

export async function inferWpsCapabilityWithModel(text = '', model = null) {
  if (!model?.providerId || !model?.modelId) return null
```

```
  const catalog = getWpsCapabilityCatalog().map(item => ({
    capabilityKey: item.capabilityKey,
    label: item.label,
    category: item.category,
    description: item.description,
    routingPromptHint: item.routingPromptHint
  }))
  const systemPrompt = [
    '你是一个 WPS 直接操作能力路由器，负责判断用户是否在要求直接调用 WPS API，而不是让 AI 生成内容。',
    '只输出合法 JSON，不要输出解释。',
    '如果用户是在要求“保存、另存为、选择路径后保存、加密、选择路径后加密、插入表格、插入空白页、插入分页符”等直接操作，请返回 capabilityKey。',
    '如果不是直接操作，请返回 capabilityKey 为空字符串。',
    'JSON 格式：{"capabilityKey":"","confidence":"high|medium|low","reason":""}'
  ].join('\n')
  const userPrompt = [
    '【用户输入】',
    String(text || '').trim(),
    '',
    '【可用直接操作能力】',
    JSON.stringify(catalog, null, 2)
  ].join('\n')
  try {
    const raw = await chatCompletion({
      providerId: model.providerId,
      modelId: model.modelId,
      temperature: 0,
      messages: [
        { role: 'system', content: systemPrompt },
        { role: 'user', content: userPrompt }
      ]
    })
    const parsed = parseJsonCandidate(raw)
    const capabilityKey = String(parsed?.capabilityKey || '').trim()
    if (!capabilityKey || !getWpsCapabilityByKey(capabilityKey)) return null
    return {
      capabilityKey,
      confidence: String(parsed?.confidence || 'low').trim() || 'low',
      reason: String(parsed?.reason || '').trim()
    }
  } catch (_) {
    return null
  }
}

export async function resolveWpsCapabilityRoute(text = '', model = null) {
  const byRule = routeWpsCapabilityByRule(text)
  if (byRule) return byRule
  return inferWpsCapabilityWithModel(text, model)
}
```